

Producer and Consumer

```
Activities  Terminal  Mar 24 16:02
midhun@localhost:~/sphore

[midhun@localhost sphore]$ ls
a          bankers    mai        Music      priority.c  sh          whrilw.sh
abc        bn          mail.c     os_lab     Public      sjf         XYZ
abc.sh     close      midhun.c   os_lab1    REC         sjf.c
ab.sh      Desktop    midhun     os_lab1.c  roundrebin  Templates
bank       Documents  midhun     os_lab.c   roundrobin.c test.sh
banker.c   Downloads  dev        priority   sample.c    Videos
                           sample.txt  while.sh

[midhun@localhost sphore]$ mkdir sphore
[midhun@localhost sphore]$ cd sphore
[midhun@localhost sphore]$ ls
[midhun@localhost sphore]$ gcc bestfit.c -o bestfit
[midhun@localhost sphore]$ ./bestfit
Produced: 0
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Consumed: 0
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
[midhun@localhost sphore]$ vi bestfit.c
[midhun@localhost sphore]$
```

```
Activities Terminal Mar 24 33:03
midhun@localhost:~/bestfit — /usr/bin/vim bestfit.c

#include <stdio.h>
#include <pehrand.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
sem_t mutex;

void *producer(void *arg)
{
    int item;
    for(int i = 0; i < BUFFER_SIZE; i++)
    {
        item = i;
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full);
    }
}

void *consumer(void *arg)
```

```
Activities Terminal Mar 24 22:04
midhun@localhost:~/bestfit — /usr/bin/vim bestfit.c

void *censumer(void *arg)
{
    int item;

    for(int i = 0; i < BUFFER_SIZE; i++)
    {
        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty);
    }
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, censumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
}
```